© copyright : spiraltrain@gmail.com

SpiralTrain
developer courses

Exercises Python Programming

Exercise 1 : Environment Configuration 2
Exercise 2 : Variables and Data Types 3
Exercise 3 : Data Structures 5
Exercise 4 : Control Flow 8
Exercise 5 : Functions 10
Exercise 6 : Modules 11
Exercise 7 : Comprehensions 12
Exercise 8 : Exception Handling 13
Exercise 9 : Python IO 14
Exercise 10 : Python Database Access 15
Exercise 11 : Python Classes 16
Exercise 12 : Optional Exercise : Python Libraries 18

Exercise 1 : Environment Configuration

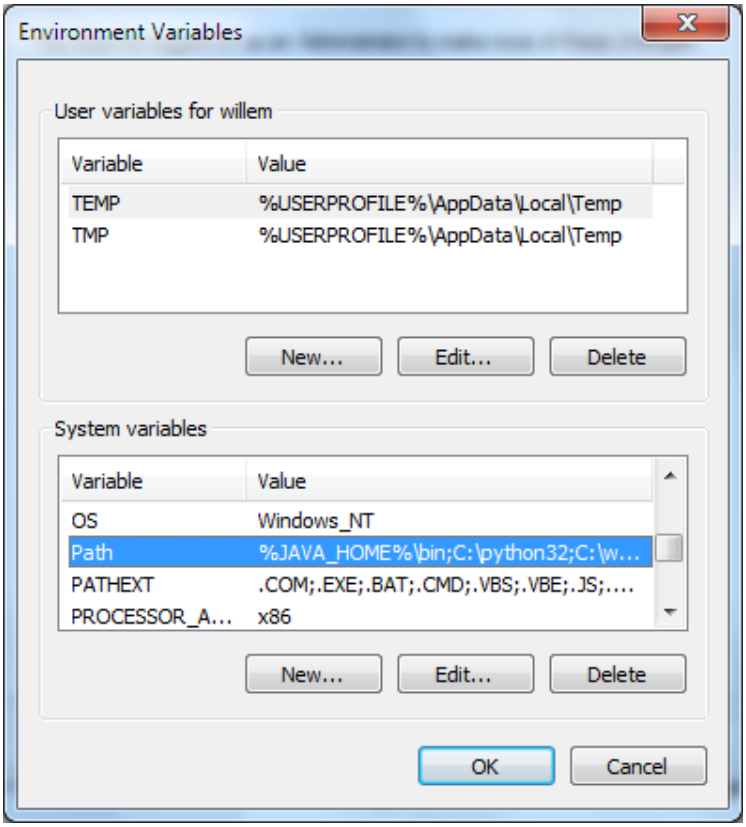
To create Python scripts we can use different IDE's :

- Pycharm Community Edition IDE that is free and can be downloaded
- Spyder IDE from the community Anaconda distribution that is free and can be downloaded
- Visual Code with a Python plugin that is free but has limited capabilities
- Eclipse IDE with the Python Plugin Pydev for Python Development, not recommended

But first we need to install a Python version.

Step 1 . Python Installation

Double click `python-3.10.7-amd64.exe` from the software directory of your `courseWare` directory and do a full install of Python in `c:\Python310`. Go to Computer->Properties->Advanced System Settings->Environment Variables and put the directory `c:\python310` on the PATH.



Step 2 : Pycharm IDE

Download and install the latest version of the Pycharm community Edition. The demos projects in the course are made with Pycharm and this is the IDE that we recommend.

Exercise 2 : Variables and Data Types

Step 1

Create a directory or project named `exercise2`. Next create a Python module `calculator2.py` in this directory and create two variables `nr1` and `nr2`. Assign integer values to them as follows :

```
nr1 = 16
nr2 = 23
```

Use your own values if you like.

Print the values, the type and the memory location of the variables with :

```
print (nr1)
print (nr2)

print (type(nr1))
print (type(nr2))

print (id(nr1))
print (id(nr2))
```

You should see that the type of the variables is `<class 'int'>`. De `id` function gives some numeric value which actually indicates the memory address in a default Python implementation.

Now change the values of the variable by assigning them to a new value and print the memory location of the variables again with the `id` function :

```
nr1 = 16
nr2 = 23
print (id(nr1))
print (id(nr2))
```

Notice that the `id` of the variables has changed just by assigning them. This illustrates the fact that integer variables are read-only. They cannot change once they are created. When you change an integer variable you actually create a new one.

Step 2

Next perform arithmetic operations on the variables by adding, subtracting, multiplying and dividing them. Store the result of these operations in other variables. An example for the addition operation is :

```
addresult = nr1 + nr2
```

Do the same for the other operations.

Display the results of the operations with an explanatory message. An example for the display of the result of the addition is :

```
# result of adding two numbers
```

```
print ("The result of the addition of : " + str(nr1) + " and " + str(nr2) +
" is : ")
print (addresult)
```

Do the same for the results of other operations. Notice the conversion function `str` that turns an integer into a string, so that it can be added to other strings.

As an alternative you can also use the string format operator `%` for printing values in string. For the explanatory string for addition this looks as follows :

```
print ("The result of the addition of : %d and %d is : " %(nr1, nr2))
```

Notice that the `str` conversion function is no longer needed. Now we need the string format operator `%` however.

A third alternative is to use the new style of formatting in Python 3 with the `format()` function. Figure out using the course manual or documentation how to use this style of string formatting.

And a fourth alternative would be to use an f string. Figure out using the course manual or documentation how to use this style of string formatting.

Also display the types of the results. The first three are of `<class 'int'>`. Notice that the type of the division is : `<class 'float'>`

Step 3

Now use the `input()` function to ask for the input of two numbers from the user as follows :

```
s1 = input("give me number 1 : ")
s2 = input("give me number 2 : ")
```

The `input()` function returns a string. Find out about the type of the return values by printing their type as follows :

```
print (type(s1))
print (type(s2))
```

Notice that the type is : `<class 'str'>`

Now perform addition with the + operator on the user input. With user input 1 and 3 the result will be 13, because 1 and 3 are seen as strings. The strings are concatenated.

```
addresult = s1 + s2     # s1=1, s2=3
print (addresult)       # prints 13
```

Next convert the string to integers with the `int()` conversion function as follows :

```
addresult = int(s1) + int(s2)
```

With the same input the result will now be 4.

When the given input cannot be converted to an `int`, a run time error occurs :

```
ValueError: invalid literal for int() with base 10: 'w'
```

Exercise 3 : Data Structures

Step 1

Create project or directory named `exercise3` and a Python module `calculator3.py`. Next create an empty list variable and ask the user for the input of two float numbers. Add these numbers, store the result in a variable and append the numbers and the result of their addition to the list.

Repeat this operation until the user quits the program by pressing the red button in the IDE. Your code could look like the following :

```
reslist = []
while True:
    f1 = float(input("Give me number 1 : "))
    f2 = float(input("Give me number 2 : "))
    addresult = f1 + f2
    reslist.append(f1)
    reslist.append(f2)
    reslist.append(addresult)
```

Also print the content of the list, print the length of the list and print a subset of the list as follows :

```
    print(f"Content of list : {reslist}"))
    print(f"Length of list : {len(reslist)}")
    print(f"Part of list : {reslist[0:3]}")
```

Finally remove the first element of the list an print the list again :

```
    print(reslist[0])
    del reslist[0]
    print(f"Content of list after deletion : {reslist}")
```

Step 2

Now create a multidimensional list in which each row contains a calculation with as elements the type of operation, the two float numbers and the result of their addition. Also calculate the results of the subtraction, multiplication and division and store these as rows in the multidimensional list. Start by allocating memory for the multidimensional list as follows :

```
multireslist = [[0,0,0,0], [0,0,0,0], [0,0,0,0], [0,0,0,0]]
```

Next ask user input with :

```
while True :
    f1 = float(input("Give me number 1 : "))
    f2 = float(input("Give me number 2 : "))
```

Then store the results of the operations `add`, `subtract`, `multiply` and `divide` in separate lists and add these list at appropriate places in the multidimensional array. For the `add` operation your code could look as follows :

```
    addresult = f1 + f2
    addlist = ["Add", f1, f2, addresult]
    multireslist[0] = addlist
```

Write similar code for the `subtract`, `multiply` and `divide` operations.

Print the multidimensional array after storing the results of the different calculations with :

```
   print (f"Content of list : {multireslist}")
```

Your result should look as follows :

```
Content of list : [['Add', 2.0, 2.0, 4.0], ['Subtract', 2.0, 2.0, 0.0],
['Multiply', 2.0, 2.0, 4.0], ['Divide', 2.0, 2.0, 1.0]]
```

Finally try to change some element in one of the sublists and notice that you are able to make the change e.g. after :

```
    multireslist[1][1] = None
```

the contents of the multidimensional list looks as follows :

```
Content of list : [['Add', 2.0, 2.0, 4.0], ['Subtract', None, 2.0, 0.0],
['Multiply', 2.0, 2.0, 4.0], ['Divide', 2.0, 2.0, 1.0]]
```

Finally change one of the sublists to a tuple e.g. :

```
addlist = ("Add", f1, f2, addresult)
```

Notice that when you try to change elements of this tuple you get the following error :

```
TypeError: 'tuple' object does not support item assignment
```

Step 3

Now create an empty dictionary. Ask the user for the input of two numbers like before. Perform add, subtract, multiply and divide operations on the numbers. Store the numbers and the results of the calculation in a small list for each calculation type. Use the following code :

```
resdict = {}
while True :
    f1 = float(input("Give me number 1 : "))
    f2 = float(input("Give me number 2 : "))

    addresult = f1 + f2
    addlist = [f1, f2, addresult]
```

Now store the list in the dictionary under the key `"Add"` as follows :

```
    resdict["Add"] = addlist
```

Use similar code for the other calculation operations. Print the dictionary after the input of two numbers. You should get the following or similar results :

```
Content of dictionary : {'Multiply': [3.0, 3.0, 9.0], 'Add': [3.0, 3.0,
6.0], 'Subtract': [3.0, 3.0, 0.0], 'Divide': [3.0, 3.0, 1.0]}
```

Try to read and extend the value at the dictionary key. The reading will succeed if the dictionary key was created before :

```
resdict["Divide"] = resdict["Divide"] + [1,0,"Infinite"]
```

But you will get an error if the key did not existed before

Next make sure that the keys "Add","Subtract", "Multiply" and "Divide" exist before starting to ask input of numbers from the user, by initializing the dictionary as follows :

```
resdict = {"Add": [], "Subtract": [], "Multiply": [], "Divide": []}
```

Now store multiple results of add calculations under the "Add" key as follows :

```
resdict["Add"] = resdict["Add"] + addlist
```

Do the same for the other calculation types. After the input of two sets of numbers you will get the following result when the dictionary is printed :

```
Content of dictionary : {'Multiply': [1.0, 1.0, 1.0, 2.0, 2.0, 4.0], 'Add':
[1.0, 1.0, 2.0, 2.0, 2.0, 4.0], 'Subtract': [1.0, 1.0, 0.0, 2.0, 2.0, 0.0],
'Divide': [1.0, 1.0, 1.0, 2.0, 2.0, 1.0]}
```

Step 4

Ask the user to input a list of integers as follows :

```
while True :
    s = input("Give me a set of numbers separated by space characters : ")
```

Now split the input string into a list with :

```
    slist = s.split(' ')
    print(slist)
```

Also sort and reverse the list with the sort and reverse functions :

```
    slist.sort()
    print(slist)
    slist.reverse()
    print(slist)
```

Finally convert the list of strings to a list of integers with :

```
    intlist = []
    for x in slist :
        x = int(x)
        intlist.append(x)
    print(intlist)
```

Feel free to experiment with other string, list and dictionary functions. Use the documentation.

Exercise 4 : Control Flow

Step 1

Create a project or directory named `exercise4` and a Python module `calculator4.py`. Copy the contents of the step 3 of the previous exercise to `calculator4.py`.
Now ask the user, beside the numbers for the calculations, also which type of calculation the user wants to perform. Use `if, else` and `elif` statements to determine which branch of the code must be entered. If the user enters "Quit" the application should stop. Your code could look like this for the part that checks whether the application should continue or not :

```
resdict = {"Add": [], "Subtract": [], "Multiply": [], "Divide": []}
while True :
    calcAction = input("What type of calculation do you want to do :")
    if calcAction == "Quit":
        break
```

For the part that checks the type of calculation to perform your code could look as follows :

```
    else :
        f1 = float(input("Give me number 1 : "))
        f2 = float(input("Give me number 2 : "))
        if calcAction == "Add":
            addresult = f1 + f2
            addlist = [f1, f2, addresult]
            resdict["Add"] = resdict["Add"] + addlist
        elif calcAction == "Subtract":
```

Step 2

Now add an option to calculate the sum and average of a list of numbers. Copy part the code from step 4 of exercise3 to convert a list of strings to a list of integers :

```
    slist = s.split(' ')
    intlist = []

    for x in slist :
        x = int(x)
        intlist.append(x)
```

Based on the input of the desired calculation type, you will have to make a distinction between the input of a list of numbers or only two numbers. Use code like the following :

```
    elif (calcAction == "Sum") or (calcAction == "Average"):
        s = input("Give a set of numbers separated by space characters : ")
```

and

```
  elif (calcAction == "Add") or (calcAction == "Subtract")
          or (calcAction == "Multiply") or (calcAction == "Divide"):
        f1 = float(input("Give me number 1 : "))
        f2 = float(input ("Give me number 2 : "))
```

Now calculate the `sum` or the `average` of the list of numbers in a `for` loop. Don't forget to declare variables for the `sum` and the `average` first. Do not use `sum` as a variable name since this is a

reserved word. Use `summation` instead. Finally print the result of the calculation. For the calculation of the sum the code could look as follows :

```
summation = 0
average = 0
if calcAction == "Sum":
   for x in slist:
      x = int(x)
      summation = summation + x
      intlist.append(x)
   print (f"Sum of list {intlist} is {summation}")
```

Step 3

Replace the `while True` statement at the beginning of the code by a test on the input of the calculation type. Stop the `while` loop when the input is `"Quit"`. Use code like the following :

```
calcAction = "Continue"
while calcAction != "Quit":
    calcAction = input("What type of calculation do you want to do :"):
```

Also add an `else` statement to the `while` loop that gets executed when the `while` loop stops. Use code like the following :

```
else:
    print("Thank you for user our software")
```

Exercise 5 : Functions

Step 1

Create a project or directory named `exercise5` and a Python module `calculator5.py`. Copy the contents of the step 3 of the previous exercise to `calculator5.py`.

Write functions to add, subtract, multiply and divide two numbers and replace the code that does the calculations inline by a call to these functions. The code for the `add` function could look as follows :

```
def add(i, j):
    z = i + j
    return z
```

and calling this function is done with :

```
addresult = add(f1, f2)
```

Step 2

Write a function `convertointlist` that converts a list of input character to a list of integers. Your code could look as follows :

```
def convertointlist(slist):
    ilist = []
    for x in slist :
        x = int(x)
        ilist.append(x)
    return ilist
```

Call this function with as parameter the input list given by the user for the sum and average calculations :

```
    s = input("Give a set of numbers separated by space characters : ")
    slist = s.split(' ')
    intlist = convertointlist(slist)
```

Also write a function `summate` that takes a list of integers as an argument, and returns the sum of the integers in that list. And write a function `average` that takes a list of integers and returns the average of these integers. For the summate function your code could look as follows :

```
def summate(ilist):
    s = 0
    for x in ilist:
        s = s + x
    return s
```

Replace the code that does the summation code and averaging code inline by a call to these functions.

Step 3

Add another calculation option to the calculator called `"Faculty"`. When the user enters this, one number should be entered and the faculty of this number should be calculated. Hereby the faculty of n is defined a `n * (n-1) * (n-2) * (n-3) * … 1`

Create a recursive function called `faculty` to perform this calculation and call this function.

Exercise 6 : Modules

Step 1

Create project or directory named `exercise6` and a Python module `calculator6.py`. Copy the contents of the `calculator5.py` from the previous exercise to `calculator6.py`.

Next create a Python module `calculations.py`. Remove the function definitions from `calculator6.py` and put them into `calculations.py`. As a result all function calls in `calculator6.py` will give errors because they are calling undefined functions.

Resolve the errors by importing the calculations module with an import statement in the first line of `calculator6.py` :

```
import calculations
```

This is not enough. You will also need to prefix all function names with the module name where they come from :

```
addresult = calculations.add (f1, f2)
```

Step 2

It is inconvenient to put the module name as prefix before the function name. Therefore there is an alternative syntax called `from <module> import <name>`. Now change to this alternative and name all the function names that must be imported in the `from import` statement :

```
from calculations import faculty, convertointlist, summate, average, add,
subtract, multiply, divide
```

As a result the prefixes before the method calls can be removed in `calculator6.py`.

Step 3

Finally make a `calcu` package. Put the `calculations.py` module in this package, Edit the `__init__.py` file and import the functions from the package in `calculator6.py`.

Exercise 7 : Comprehensions

Step 1 :

Create project or directory named `exercise7` and a Python module `calculator7.py`. Copy the contents of the `calculator6.py` from the previous exercise to `calculator7.py`. Do not copy the `calcu` package.
Now we will replace the functions from the `calcu` package by functional programming constructs like lambda's, comprehensions and reduce functions.
Put the import `from functools import reduce` in the first line of `calculator7.py`.

Step 2 :

Replace the following code by a list comprension :

```
s = input("Give a set of numbers separated by space characters : ")
slist = s.split(' ')
intlist = convertointlist(slist)
```

We do not need a `convertointlist` function because we can do the conversion in a one liner using a list comprehension.

Step 3 :

Replace following code by a call to the `reduce` function which uses a `lambda` function for summing :

```
summation = summate(intlist)
print(f"Sum of list {intlist} is {summation}")
```

We do not need a `summate` function because we can calculate the sum in a one liner using the `reduce` function and a `lambda`.

Step 4:

Replace following code by a call to the `reduce` function which uses a `lambda` function for summing and divide by the length of the list of integers :

```
av = average(intlist)
print(f"Average of list {intlist} is {av}")
```

We do not need a `average` function because we can calculate the average in a one liner using the `reduce` function, a `lambda` and the length of the list with the `len` function.

Step 5:

Replace the following code with one line which uses the definition and call of a lambda function for `add`. Do the same for `subtract, divide` and `multiply`. All can be replaced by one liners which do not use an external function.

```
addresult = add (f1, f2)
addlist = [f1, f2, addresult]
resdict["Add"] = resdict["Add"] + addlist
```

Step 6 :

Finally replace the calculation of the faculty with the following code and observe that is gives the correct results :

```
n = int(input("Give a positive integer number : "))
fac = reduce((lambda x, y : x * y), list(range(1, n+1)))
```

© copyright : spiraltrain@gmail.com

Exercise 8 : Exception Handling

Step 1

Create project or directory named `exercise8` and a Python module `calculator8.py`. Copy the contents of the `calculator6.py` from the previous exercise to `calculator8.py`. Also copy the `calcu` package which now should contain comprehensions.

Write `try: except:` statements for the following cases. In all these cases the current program crashes. With exception handling we can ask for new input.

1) Division by zero.
2) Wrong input for Add, Subtract, Multiply and Divide operations.
3) Wrong input for the number for the factorial
4) Wrong input for the list of integers.

Step 2

Define an exception class. Then write a `try:except:` statement in which you throw and catch that specific exception. Use code from the demo directories or the internet as example code.

Step 3

The code that we have made up and to now is not very clean. We should make our script more maintainable and extensible. One big script in which everything happens is not so good. The `if` statements are also nested too much.

Now refactor the code so it consists of a main from where three functions are called namely : `OneNumberCalculations`, `TwoNumberCalculations` and `MultipleNumberCalculations`. Each of these functions should get a `CalcAction` string as parameter.

Write this script in the file `calculatorclean.py`.

Make sure the script works exactly the same as the non-clean version.

Exercise 9 : Python IO

Step 1

Create project or directory named `exercise9` and a Python module `calculator9.py`. Copy the contents of the `calculator8.py` from the previous exercise to `calculator9.py`. Also copy the `calcu` package. You will use the solution code of the previous exercise as a starting point.

Open a text file for writing just before entering the while loop that asks the user to choose a calculation type. Use the following code :

```
fw = open('calculationdata.txt', mode='w', encoding='UTF-8')
```

Next add the details of a calculation to a line in this file when each calculation is performed.

For the `Faculty` calculation you could use the following code :

```
fw.write(f"{calcAction},{x},{faculty(x)}\n")
```

For the `Sum` and `Average` calculations you could use the following code :

```
fw.write(f"{calcAction},{intlist},{summate(intlist)}\n")
```

And for the `Add`, `Subtract`, `Divide` and `Multiply` calculations you could use the following code just after the input of the numbers :

```
f1 = float(input("Give me number 1 : "))
f2 = float(input("Give me number 2 : "))
fw.write(f"{calcAction},{f1},{f2}")
```

And the following code just after each calculation :

```
fw.write(f"{add(f1, f2)}\n")
```

Finally close the file when you give the `"Quit"` command otherwise nothing is written.

```
fw.close()
```

Now enter some calculations to test your code and observe the contents of the file.

Step 2

Next create a new Python module `readcalculations.py`, read the lines of the file and display its contents with the following code :

```
fr = open('calculationdata.txt', mode='r', encoding='UTF-8')
for line in fr.readlines():
    s = line.rstrip('\n')
    slist = s.split(',')
    print (slist)
fr.close()
```

The results could look as follows :

```
['Faculty', '12', '479001600\n']
['Sum', '[1', ' 1', ' 3]', '5\n']
['Average', '[4', ' 5', ' 6]', '5.0\n']
```

Exercise 10 : Python Database Access

Step 1

Create project or directory named `exercise10` and a Python module `calculator10.py`. Copy the contents of the `calculator9.py` from the previous exercise to `calculator10.py`. Also copy the `calcu` package. You will use the solution code of the previous exercise as a starting point and replace the file access code with database access code.

Before entering the `while` loop in which the user is asked to choose the type of calculation he wants to perform use the following code to create a database `calculations.db` and a table `Calculations` :

```
con = sqlite3.connect('calculations.db')

with con:
    cur = con.cursor()
    cur.execute("DROP TABLE IF EXISTS Calculations")
    cur.execute("CREATE TABLE Calculations
                (Id VARCHAR(10), Nr1 INT, Nr2 INT, Result DECIMAL)")
```

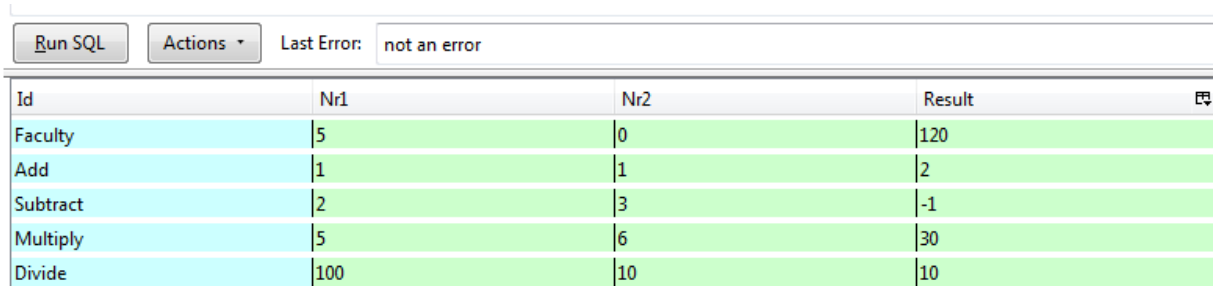
Next consider the Faculty calculation type and replace the code where data is written to the file with code that writes data to the database. Write the following code on the same line :

```
with con:
     cur = con.cursor()
     cur.execute("INSERT INTO Calculations ('Id', 'Nr1', 'Nr2', 'Result')
                 VALUES(?,?,?,?)", (calcAction, x, 0, faculty(x)))
```

In this exercise we will not consider the Sum and Average calculation types. Just remove the code that writes data to the file. We will however consider the Add, Subtract, Divide and Multiply calculation types. For these calculation types replace the file access code with the following database access code. For instance for the Add calculation type :

```
with con:
     cur = con.cursor()
     cur.execute("INSERT INTO Calculations ('Id', 'Nr1', 'Nr2', 'Result')
                 VALUES(?,?,?,?)", (calcAction, f1, f2, add(f1, f2)))
```

Next insert some records in the database, observe the existence of the `calculations.db` file in the directory of the script and use the SQLiteStudio Database Viewer to connect to SQLite and examine the contents of the database. The table contents could look as follows



Id	Nr1	Nr2	Result
Faculty	5	0	120
Add	1	1	2
Subtract	2	3	-1
Multiply	5	6	30
Divide	100	10	10

Also make a script to read the database records back in and display them in the console.

Exercise 11 : Python Classes

Step 1 : Employee class

Make an `Employee` class representing some employee working in a company in the python module `employee.py`. The `Employee` class should have two attributes representing the name and the ID of an employee.

Add a constructor to the `Employee` class which sets the attributes when they are passed. Make a simple program which declares an `Employee` object and display the values of the attributes of the `Employee` object by accessing the attributes directly. Use code like the following :

```
class Employee(object) :
    def __init__(self, name):
        print("Calling Employee constructor")
        self.name = name
```

Next make the attributes private in the Employee class and add public set and get member functions in the Employee class to be able to access these attributes. Change the values of the attributes through the accessor functions and display their changed values. Use code like the following :

```
    def set_name(self, value):
        self.__name = value
    def del_name(self):
        del self.__name
```

Notice that private data cannot by accessed directly from outside of the class. This demonstrates the object oriented principle of encapsulation. In Python there is a workaround so you can still access private data. Use this workaround also for demonstration purpose.

Also add a destructor to the class and keep a class variable which counts how many instances of the class are present. Use code like the following :

```
    def __del__(self):
        class_name = self.__class__.__name__
        print (class_name, "destroyed")
```

Write print statements when the constructor or destructors and the getters and setters are called. Also print an instance variable directly. What is the result?

Next make a property in the class with the following code :

```
  name = property(get_name, set_name, del_name, "name's docstring")
```

Access the `Employee` object through the property as follows :

```
    e1.name = "Some Name"
    print(e1.name)
```

Also write statements to show the class and object data :

```
    print("Employee.__doc__:", Employee.__doc__)
    print("Employee.__name__:", Employee.__name__)
```

```
    print("Employee.__module__:", Employee.__module__)
    print("Employee.__bases__:", Employee.__bases__)
    print("Employee.__dict__:", Employee.__dict__)
```

Put the Employee class in a script in the employees package and add code to be able to export the Employee class from this package.

Give the Employee class a `calcSal()` method as follows :

```
    def calcSal(self):
        return 0
```

Optionally you can also add a `hiringdate` attribute to the `Employee` class.

Add the following code to test the `Employee` class in stand alone mode :

```
if __name__ == '__main__':
    main()
```

Step 2 : Company Script

Make a Company script above the `employees` package.

Add the necessary import statements to be able to find the `Employee` class in the `employee` script in the `employees` package. If you keep the `__init__.py` file in the package empty your code could look as follows :

```
import employees.employee

e1 = employees.employee.Employee("Albert Einstein")
```

Create a number of employees and put them in a list, called for instance `employeeList`. Next iterate through the list and print the name of the employees in the list.

If time permits :

Alternatively you could create a `Company` class with functions to add or to remove employees from the `employeeList` which displays a list of employees including their name and ID. In that case make a simple script which declares a `Company` object, add several employees to the company and display a list of employees.

The relationship between the `Company` class and the `Employee` class is an association relation of the normal type. We can say every `Company` employs `Employees` or every `Employee` works for a `Company`.

© copyright : spiraltrain@gmail.com

Exercise 12 : Optional Exercise : Python Libraries

See PDF in Appendix